DSAPS Assignment 3

Number Theory: Solutions and Explanations

Section 3 · In-lab contest, 2 November 2026

This guide explains the four problems of the Number Theory section. Each problem has the same parts: **the task**, **the idea**, an **example**, **C++ code** and **common mistakes**. All code was tested on the real test data and passes every batch.

Problem	Topic	Marks
1. Divisible by Either	gcd, lcm, avoiding overflow	10
2. Number Toolkit	sieve, smallest prime factor, factorisation	30 (3 parts × 10)
3. Counting Toolkit	factorials, nCr, modular inverse	30 (3 parts × 10)
4. Soldier Count	Chinese Remainder Theorem	30 (20 + 10)

Tools used in several problems

1. Work modulo $p = 10^9 + 7$. Take the remainder after **every** multiplication. Two numbers below p multiply to less than about 10^{18}, which fits in a `long long`. A `long long` holds up to about 9.2×10^{18}.

2. Binary exponentiation computes x^e mod m in about log(e) steps. It squares the base and uses the bits of e:

```
ll power(ll base, ll exp, ll m) {
    ll result = 1;
    base %= m;
    while (exp > 0) {
        if (exp & 1) result = result * base % m;
        base = base * base % m;
        exp >>= 1;
    }
    return result;
}
```

3. Modular inverse. You cannot divide under a modulus. Instead you multiply by the inverse. If p is prime, the inverse of x is x^{p-2} mod p (Fermat's little theorem). If the modulus is not prime, use the extended Euclidean algorithm (Problem 4).

4. Fast input and output. Use `ios::sync_with_stdio(false); cin.tie(nullptr);` and print `"\n"` instead of `endl`. With up to 3×10^5 lines, `endl` is slow.

Problem 1: Divisible by Either — 10 marks

The task. For each query (N, a, b), count how many numbers from 1 to N are divisible by a **or** by b.

The idea

Count the multiples of each number, then fix the double counting.

- Multiples of a up to N: N / a. Multiples of b up to N: N / b.
- A number divisible by **both** was counted twice. Such numbers are exactly the multiples of lcm(a, b).
- So subtract them once: **answer = N / a + N / b − N / lcm(a, b)**.
- Find g = gcd(a, b) with Euclid's algorithm. Then lcm = a / g × b.

Why divide first? a × b can be as large as 10^{36}, which overflows. But a / g × b is the lcm itself, and the problem guarantees it is at most 9 × 10^{18}, so it fits in a `long long`.

Example

N = 100, a = 4, b = 6. Multiples of 4: 100 / 4 = 25. Multiples of 6: 100 / 6 = 16.
gcd = 2, so lcm = 12. Multiples of 12: 100 / 12 = 8.
Answer = 25 + 16 − 8 = **33**.

Common mistakes

- Computing `a * b / g`. The product a × b can overflow even when the lcm fits.
- Using `int` for N, a, b, or for the remainder inside the gcd. Use `long long` everywhere.
- Forgetting to subtract N / lcm (this counts some numbers twice).
- Finding the lcm by looping, or the gcd by repeated subtraction. Both are too slow for values up to 10^{18}.

Code

```
#include <iostream>
using namespace std;
typedef long long ll;

// Euclid's algorithm
ll my_gcd(ll a, ll b) {
    while (b != 0) {
        ll r = a % b;
        a = b;
        b = r;
    }
    return a;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    cin >> q;
    while (q--) {
        ll n, a, b;
        cin >> n >> a >> b;

        ll g = my_gcd(a, b);
        // divide first, then multiply: the lcm (at most 9e18) fits in long long
        ll lcm = a / g * b;

        cout << n / a + n / b - n / lcm << "\n";
    }
    return 0;
}
```

Time: O(log(min(a, b))) per query.

Problem 2: Number Toolkit — 30 marks

The task. All queries in one input have the same type t, with numbers up to 10^7 and up to 3×10^5 queries. **Part a (t = 1):** print the prime factors of x as pairs "p e". **Part b (t = 2):** count the primes in [L, R]. **Part c (t = 3):** count the divisors of x.

The idea

Build **one sieve** that stores the **smallest prime factor** of every number, called `spf[x]`. All three parts use it.

1. **Build spf.** For each prime i (when spf[i] is still 0), set spf[i] = i. Then for the multiples of i, starting at i × i, set spf[j] = i if it is still 0. A number i is prime exactly when spf[i] = i.
2. **Part a: factorise.** Take p = spf[x]. Divide x by p again and again, counting how many times (the exponent e). Print p and e. Repeat with the new x until x = 1. This takes only about log(x) steps, because x gets smaller each time. The primes come out in increasing order.
3. **Part c: divisors.** If $x = p_1^{e1} \times p_2^{e2} \times \ldots$, each prime can be used 0, 1, …, e times. So the number of divisors is **(e1 + 1) × (e2 + 1) × …**
4. **Part b: count primes.** Make a prefix array: pi[i] = pi[i−1] + (1 if i is prime). The answer is **pi[R] − pi[L−1]**. Use L−1, so that L itself is counted when it is prime. The number 1 is not prime.

Example

x = 360 = $2^3 \times 3^2 \times 5$. Part a prints **2 3 3 2 5 1**. Part c: (3+1)(2+1)(1+1) = **24**.
Part b with L = 10, R = 100: there are 25 primes up to 100 and 4 up to 9, so the answer is 25 − 4 = **21**.

Common mistakes

- Trial division for every query (up to about 3000 steps each). It is too slow for 3×10^5 queries.
- Writing `j = i * i` with `int`. For i above 46340, i × i overflows and the program crashes. Use `long long`.
- Storing the factor list of every number up to 10^7. This uses far more than the memory limit. Factorise each query when it arrives.
- Using pi[L] instead of pi[L−1], or counting 1 as a prime.
- Printing primes in the wrong order, or printing "p^e" instead of "p e".

Code

```
#include <iostream>
#include <vector>
using namespace std;

const int N = 10000000;
vector<int> spf(N + 1, 0);       // spf[x] = smallest prime factor of x
vector<int> pi_cnt(N + 1, 0);    // pi_cnt[x] = number of primes <= x

void build() {
    for (int i = 2; i <= N; i++) {
        if (spf[i] == 0) {       // i is prime
            spf[i] = i;
            // long long, because i * i can overflow int
            for (long long j = (long long)i * i; j <= N; j += i)
                if (spf[j] == 0) spf[j] = i;
        }
    }
    for (int i = 1; i <= N; i++)
        pi_cnt[i] = pi_cnt[i - 1] + (spf[i] == i ? 1 : 0);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    build();

    int t, q;
    cin >> t >> q;
    while (q--) {
        if (t == 2) {
            // part b: primes in [l, r]
            int l, r;
            cin >> l >> r;
            cout << pi_cnt[r] - pi_cnt[l - 1] << "\n";
        } else {
            // parts a and c: factorise x using spf
            int x;
            cin >> x;
            int divisors = 1;
            bool first = true;
            while (x > 1) {
                int p = spf[x], e = 0;
                while (x % p == 0) {
                    x /= p;
                    e++;
                }
                divisors *= e + 1;
                if (t == 1) {
                    // part a: print "p e"
                    if (!first) cout << " ";
                    cout << p << " " << e;
                    first = false;
                }
            }
            if (t == 1) cout << "\n";
            else cout << divisors << "\n";    // part c
        }
    }
    return 0;
}
```

Time: Sieve: about N log log N for N = 10^7. Each query: O(log x) for parts a and c, O(1) for part b. It uses about 80 MB.

Problem 3: Counting Toolkit — 30 marks

The task. Answer C(n, r) mod p, where p = $10^9 + 7$. **Part a (t = 1):** print C(n, r), or 0 if r > n (n, r up to 10^6, up to 10^5 queries). **Part b (t = 2):** print C(n,0) + C(n,1) + … + C(n,r) (up to 50 queries). **Part c (t = 3):** print C(n, r) when n is up to 10^9 but r is at most 10^6 (up to 50 queries).

The idea

C(n, r) = n! / (r! × (n−r)!). We cannot divide under a modulus, so we multiply by the **inverse** of the bottom part.

1. **Precompute once.** fact[i] = i! mod p for i up to 10^6. Then inv_fact[i] = 1 / i! mod p. Get the last one with Fermat: inv_fact[10^6] = power(fact[10^6], p−2). Then go backwards: **inv_fact[i−1] = inv_fact[i] × i**. This needs only one power call.
2. **Part a.** C(n, r) = fact[n] × inv_fact[r] × inv_fact[n−r], each step mod p. Print 0 when r > n. O(1) per query.
3. **Part b.** Loop i from 0 to r and add C(n, i) from part a. O(r) per query.
4. **Part c.** n is too big for a table. But C(n, r) = (n × (n−1) × … × (n−r+1)) / r!. Loop r times to multiply the top part (mod p), then multiply by inv_fact[r]. O(r) per query. This is correct because n < p, so no factor on top is a multiple of p.

Example

C(5, 2) = 5! / (2! × 3!) = 120 / 12 = **10**.
Part b: n = 5, r = 2 gives 1 + 5 + 10 = **16**. If r = n, the sum is 2^n.
Part c: C(10^9, 2) = 10^9 × (10^9 − 1) / 2, which is 28 mod p.

Common mistakes

- Not taking mod inside the loop of part c (the product overflows).
- Dividing the reduced numerator by r! with ordinary division. It gives wrong numbers. Always multiply by the inverse.
- Making a modular power for every term in part b. That is about 30 times slower and gets TLE. Use the tables.
- Building a factorial table of size n in part c (n can be 10^9).
- Forgetting that part a must print 0 when r > n, or using `int` for n and r.

Code

```
#include <iostream>
#include <vector>
using namespace std;
typedef long long ll;

const ll MOD = 1000000007;
const int MAXN = 1000000;

// base^exp mod MOD (binary exponentiation)
ll power(ll base, ll exp) {
    ll result = 1;
    base %= MOD;
    while (exp > 0) {
        if (exp & 1) result = result * base % MOD;
        base = base * base % MOD;
        exp >>= 1;
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // fact[i] = i!   and   inv_fact[i] = 1 / i!   (both modulo MOD)
    vector<ll> fact(MAXN + 1), inv_fact(MAXN + 1);
    fact[0] = 1;
    for (int i = 1; i <= MAXN; i++) fact[i] = fact[i - 1] * i % MOD;
    // Fermat: x^(MOD-2) is the inverse of x
    inv_fact[MAXN] = power(fact[MAXN], MOD - 2);
    for (int i = MAXN; i >= 1; i--) inv_fact[i - 1] = inv_fact[i] * i % MOD;

    int t, q;
    cin >> t >> q;
    while (q--) {
        ll n, r;
        cin >> n >> r;
        if (t == 1) {
            // part a
            if (r > n) cout << 0 << "\n";
            else cout << fact[n] * inv_fact[r] % MOD * inv_fact[n - r] % MOD << "\n";
        } else if (t == 2) {
            // part b: C(n,0) + C(n,1) + ... + C(n,r)
            ll sum = 0;
            for (ll i = 0; i <= r; i++)
                sum = (sum + fact[n] * inv_fact[i] % MOD * inv_fact[n - i]) % MOD;
            cout << sum << "\n";
        } else {
            // part c: n up to 1e9, so multiply n * (n-1) * ... * (n-r+1)
            ll num = 1;
            for (ll i = 0; i < r; i++) num = num * ((n - i) % MOD) % MOD;
            cout << num * inv_fact[r] % MOD << "\n";
        }
    }
    return 0;
}
```

Time: Precomputation $O(10^6)$. Part a: O(1) per query. Parts b and c: O(r) per query, so at most about 50×10^6 steps.

Problem 4: Soldier Count (CRT) — 30 marks

The task. Find the smallest $x \geq 0$ with $x \equiv a_i \pmod{m_i}$ for every i. The m_i are pairwise coprime, $k \leq 15$, $m_i \leq 10^9$, and their product is at most 10^{18}. In part a all m_i are primes. In part b they need not be.

The idea

Build the answer **one condition at a time**.

1. Start with $x = a_1$ and $M = m_1$. Now x satisfies the first condition. Every number $x + M \times t$ also satisfies it, for any t.
2. To add the next condition $x \equiv a \pmod{m}$, choose t so that $\mathbf{x + M \times t \equiv a \pmod{m}}$. Solving for t gives **t = ((a − x) mod m) × inv(M mod m, m) mod m**, where inv is the modular inverse. It exists because m shares no factor with M.
3. Update **x = x + M × t** and **M = M × m**. After all conditions, x is the smallest non-negative answer, because $x < M$.
4. **Part a (all m prime):** the inverse of v mod m is $v^{m-2} \bmod m$ (Fermat), using the power function from page 1: `power(M % m, m - 2, m)`.
5. **Part b (m not always prime):** use the **extended Euclidean algorithm**. It finds x, y with $a \times x + m \times y = 1$. Then $a \times x \equiv 1 \pmod{m}$, so the inverse is x mod m. This works for any modulus, so the code below passes **both** parts.

No overflow: diff × inv is below $m^2 \leq 10^{18}$, and $M \times t$ is below $M \times m \leq 10^{18}$. Both fit in a `long long`.

Example

$x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$, $x \equiv 2 \pmod{7}$.
Start: $x = 2$, $M = 3$. Next, $2 + 3t \equiv 3 \pmod{5}$, so $3t \equiv 1$ and $t = 2$ (the inverse of 3 mod 5 is 2). Now $x = 8$, $M = 15$.
Next, $8 + 15t \equiv 2 \pmod{7}$. Since $15 \equiv 1$ and $2 - 8 \equiv 1 \pmod{7}$, $t = 1$. Now $x = 23$, $M = 105$.
The answer is **23**.

Common mistakes

- Not making (a − x) non-negative. In C++ the remainder can be negative, so add m and take the remainder again.
- Using `int` for diff or the inverse. Their product overflows.
- Using Fermat's inverse when m is not prime. This works in part a only.
- Searching for x by trying x, $x + m_1$, $x + 2m_1$, ... This is far too slow.
- Printing M instead of 0 when all $a_i = 0$. The answer 0 is correct.

Code

```
#include <iostream>
#include <vector>
using namespace std;
typedef long long ll;

// finds x, y with a*x + b*y = gcd(a, b)
ll ext_gcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    ll x1, y1;
    ll g = ext_gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

// inverse of a modulo m (works when gcd(a, m) = 1)
ll mod_inv(ll a, ll m) {
    ll x, y;
    ext_gcd(a % m, m, x, y);
    return ((x % m) + m) % m;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int k;
        cin >> k;
        vector<ll> a(k), m(k);
        for (int i = 0; i < k; i++) cin >> a[i] >> m[i];

        // x solves the first i conditions, modulo M
        ll x = a[0], M = m[0];
        for (int i = 1; i < k; i++) {
            // (a_i - x) mod m_i, never negative
            ll diff = ((a[i] - x % m[i]) % m[i] + m[i]) % m[i];
            // choose t so that x + M*t == a_i (mod m_i)
            ll t = diff * mod_inv(M % m[i], m[i]) % m[i];
            x += M * t;
            M *= m[i];
        }
        cout << x << "\n";
    }
    return 0;
}
```

Time: O(k log m) per test case.

Quick checklist for the lab

- Use `long long` for anything that can pass 2×10^9.
- Take mod after every multiplication, and never subtract without adding the modulus back.
- Precompute tables once, before reading queries.
- Check the limits: how big can the numbers be, and how many queries are there?
- Test the edge cases: 0, 1, the largest values, and the case r > n.